Amazon SDE II System Design

Deep Dive: URL Shortener and Rate Limiter

An elaborated study guide for the first two questions of the Top 5 guide: plain-language explanations, worked examples, diagrams and the High-Level Design (HLD) you can draw and explain in the interview.

What is inside

Chapter	You will be able to
Question 1: URL shortener	Explain the read-heavy shape of the problem, size it with arithmetic, generate unique 7-character codes without a bottleneck, draw the HLD with separate read and write paths, and defend 302, caching and alias decisions
Question 2: Rate limiter	Explain and compare five algorithms with worked numbers, write the token bucket in Java, explain why distributed enforcement needs an atomic Redis script, and draw and narrate the gateway HLD with failure handling

The 45-minute shape every answer should follow

Minutes	Step	What to produce
0 to 5	Clarify	Requirements, scale, assumptions stated aloud
5 to 10	Estimate	QPS, storage, cache size, with the arithmetic visible
10 to 15	API and data model	Endpoints, keys, example rows
15 to 25	High-Level Design	The diagram, then a walk through each flow
25 to 40	Deep dives	The two or three hardest decisions with trade-offs
40 to 45	Wrap-up	Failures, monitoring, what you would do next

> **Amazon-specific tip.** Tie decisions back to the customer (for example: "a broken short link is a broken customer experience, so the redirect path is the one I protect most"), name the bottleneck before the interviewer does, and mention operations (metrics, alarms, what you would monitor). Numbers in this guide are assumptions chosen to demonstrate the method; replace them with the interviewer's numbers.

Question 1: Design a URL Shortener

> **How to use this chapter.** Part A explains the problem in plain language. Part B builds the numbers. Part C is the High-Level Design (HLD) you will draw and talk through in the interview. Part D goes deep on the hard decisions with worked examples. Part E is rehearsal material: follow-ups, mistakes and a one-page cheat sheet.

Part A. Understand the problem first

What the product does

A URL shortener turns a long link into a short one and sends people from the short link to the long one. Example: a customer has `https://www.example.com/products/electronics/phones?id=88231&ref=newsletter&utm_source=email` and wants something that fits in a text message. The service gives back `https://sho.rt/aZ3k9Q`. When anybody opens `sho.rt/aZ3k9Q`, their browser is redirected to the long address.

The whole system is two operations: **create** a mapping (code to long URL) and **redirect** by looking the code up. Everything else in the interview (caching, ID generation, analytics, expiry) is about doing those two operations at scale and reliably.

> **Key idea.** Reads vastly outnumber writes. A link is created once but clicked many times. That 100:1 (or higher) read/write ratio is the single fact that shapes the whole design: make the read path extremely fast and cheap, and accept more work on the write path.

Vocabulary you should use

Term	Meaning
Short code (key, slug)	The 6 to 8 characters after the domain, for example `aZ3k9Q`.
Long URL	The original destination the user wants to share.
Mapping	The stored pair: short code to long URL (plus owner, created time, expiry).
Redirect	An HTTP response (301 or 302) with a `Location` header telling the browser where to go.
Custom alias	A code the user picks (`sho.rt/summer-sale`) instead of a generated one.
Read path / write path	Read path = redirect (hot, 100x more traffic). Write path = create (cold).

Part B. Requirements and numbers

B1. Clarifying questions (spend the first 3 to 5 minutes here)

Question to ask	Why it matters	Assumption to state
How many new URLs per month? Read to write ratio?	Drives storage, QPS and cache sizing	100 million new URLs per month, 100 reads per write
How long should a short link live?	Drives storage growth and the expiry feature	Default: never expires; optional expiry set by the user
Custom aliases needed?	Adds a uniqueness race to the write path	Yes, optional
Do we need click analytics?	Adds an asynchronous pipeline	Yes, but eventually consistent (minutes of delay is fine)

Question to ask	Why it matters	Assumption to state
Can a link be edited or deleted?	Affects caching (invalidation) and 301 vs 302	Delete yes; edit of destination no (create a new link)
Availability and latency target?	Drives replication and cache design	Redirects: 99.99% available, p99 under 50 ms server time

> **Say it like this:** "Before I design, let me confirm scope. I will assume 100 million new URLs a month, a 100 to 1 read to write ratio, optional custom aliases and expiry, and click analytics that can lag by a few minutes. I will make the redirect path highly available and fast, and I will not cover user accounts or link editing. Does that match what you have in mind?"

B2. Back-of-the-envelope estimates, step by step

Interviewers want to see the method, not exact numbers. Round aggressively and say each step aloud. A month has about 2.6 million seconds (30 x 24 x 3600 = 2,592,000), and a day has 86,400 seconds.

Quantity	Working	Result
Write rate	100,000,000 new URLs / 2.6 million seconds	about 40 writes per second on average
Peak writes	Assume peak is 10x average	about 400 writes per second
Read rate	40 writes/s x 100 reads per write	about 4,000 redirects per second on average
Peak reads	10x average	about 40,000 redirects per second
Records in 5 years	100 million x 12 months x 5 years	6 billion mappings
Record size	code (7 B) + long URL (about 200 B) + owner, timestamps, expiry, overhead	about 500 bytes
Storage in 5 years	6 billion x 500 bytes = 3,000 GB	about 3 TB (small for a modern database)
Redirects per day	4,000 per second x 86,400	about 345 million per day
Cache size (80/20 rule)	Cache the hottest 20%: 0.2 x 345 million x 500 bytes	about 35 GB (fits in a few cache nodes)

> **What the numbers tell you.** 40,000 requests per second at peak is easy for a cache-fronted key-value store, but it would stress a single relational database. Total storage of 3 TB is small, so the challenge is not size; it is **read latency, availability and unique code generation**. Say this out loud; it shows you used the math to make decisions.

B3. How long must the code be?

Each character comes from 62 symbols (a-z, A-Z, 0-9). With n characters you can represent 62^n different codes.

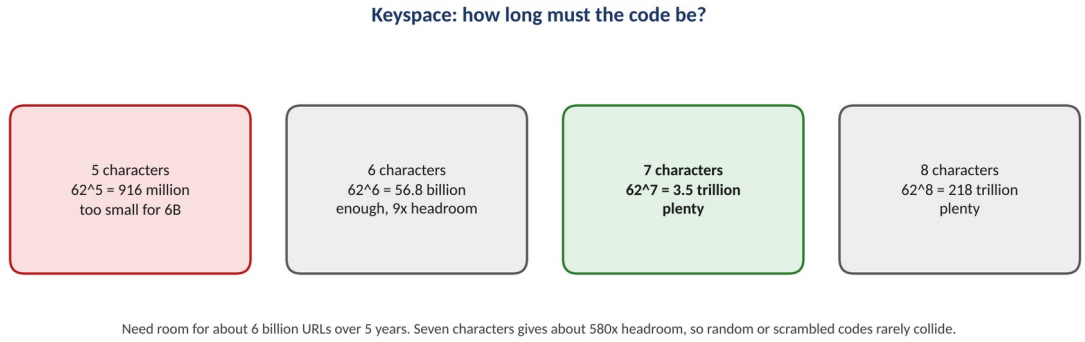


Figure 1. Keyspace for 5 to 8 character codes

Six billion records need a keyspace much larger than 6 billion, otherwise random codes collide constantly. At 7 characters the keyspace is about 3.5 trillion, so 6 billion used codes fill only 0.17 percent of it. That means a random 7-character code collides with an existing one only about 1 time in 590 inserts, and a collision is handled with a retry. This is why **7 characters** is the standard answer.

B4. API design with examples

```
POST /v1/urls
  Request:  { "longUrl": "https://www.example.com/very/long/path?x=1",
              "customAlias": "summer-sale",     // optional
              "expiresAt": "2027-01-31T00:00:00Z" }   // optional
  Response: 201 Created
            { "shortUrl": "https://sho.rt/summer-sale", "code": "summer-sale" }
  Errors:   400 invalid URL | 409 alias already taken | 429 too many creates

GET /{code}
  Response: 302 Found          Location: https://www.example.com/very/long/path?x=1
  Errors:   404 unknown code | 410 Gone (expired or deleted)

DELETE /v1/urls/{code}
  Response: 204 No Content (only the owner may delete)
```

Notice the API choices: create is a POST that returns **201 and the new short URL**; the redirect is a GET on the root path so links stay short; **409 Conflict** is the honest answer when two people want the same alias; **410 Gone** tells crawlers and browsers that a code existed but is permanently unavailable, which is more precise than 404.

B5. Data model with example rows

code (partition key)	longUrl	ownerId	createdAt	expiresAt (TTL)
aZ3k9Q	https://www.example.com/products/phones?id=88231	user-501	2026-10-05T09:12:00Z	(none)
summer-sale	https://www.example.com/sale	user-77	2026-10-05T09:15:30Z	1801007999
8sudW	https://news.example.org/story/42	user-12	2026-10-06T14:00:10Z	1790000000

Access pattern: **only ever look up by code**, so the code is the partition key. No joins, no range scans on the hot path. That is why a key-value store like DynamoDB fits: single-digit-millisecond lookups, horizontal scaling and a built-in TTL feature for expiry. A secondary index on `ownerId` supports "list my links" without touching the redirect path.

Part C. High-Level Design (HLD)

This is the diagram to draw on the whiteboard. Draw it left to right: client, front door, two services, then storage. Keep read and write paths visibly separate; that single drawing choice communicates that you understand the 100:1 ratio.

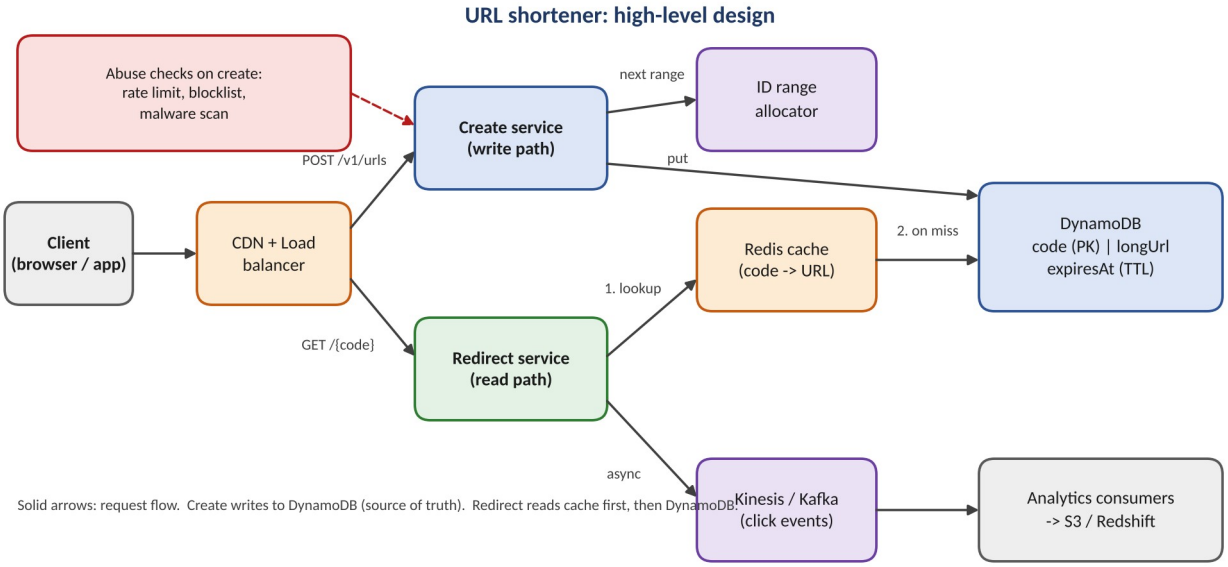


Figure 2. URL shortener HLD (write path on top, read path below)

C1. Components and why each exists

Component	Responsibility	Technology example	Why it is there
CDN and load balancer	Terminate TLS, spread traffic, optionally cache popular redirects at the edge	CloudFront + ALB	Absorbs viral links before they reach servers
Create service	Validate URL, enforce limits, generate or claim a code, store the mapping	Stateless containers (ECS/EKS/Lambda)	Stateless, so it scales by adding instances
Redirect service	Look up the code, return 302, emit a click event	Stateless containers	Separate from create so the hot path scales and fails independently
ID range allocator	Hands each Create instance a block of unique numbers	DynamoDB atomic counter item	No global lock per request, no collisions
Cache	Hold the hottest code to URL mappings in memory	ElastiCache for Redis	Takes about 90 to 95 percent of reads off the database
Mapping database	Source of truth for all mappings	DynamoDB (code as partition key)	Key-value lookups at any scale, TTL, multi-AZ replication
Event stream and analytics	Count clicks without slowing redirects	Kinesis or Kafka, then S3/Redshift	Moves slow work off the request path
Abuse protection	Rate limit creation, block malicious targets	Rate limiter (Question 2), blocklist, scanner	Shorteners are a favorite tool of phishers

C2. The write path in motion

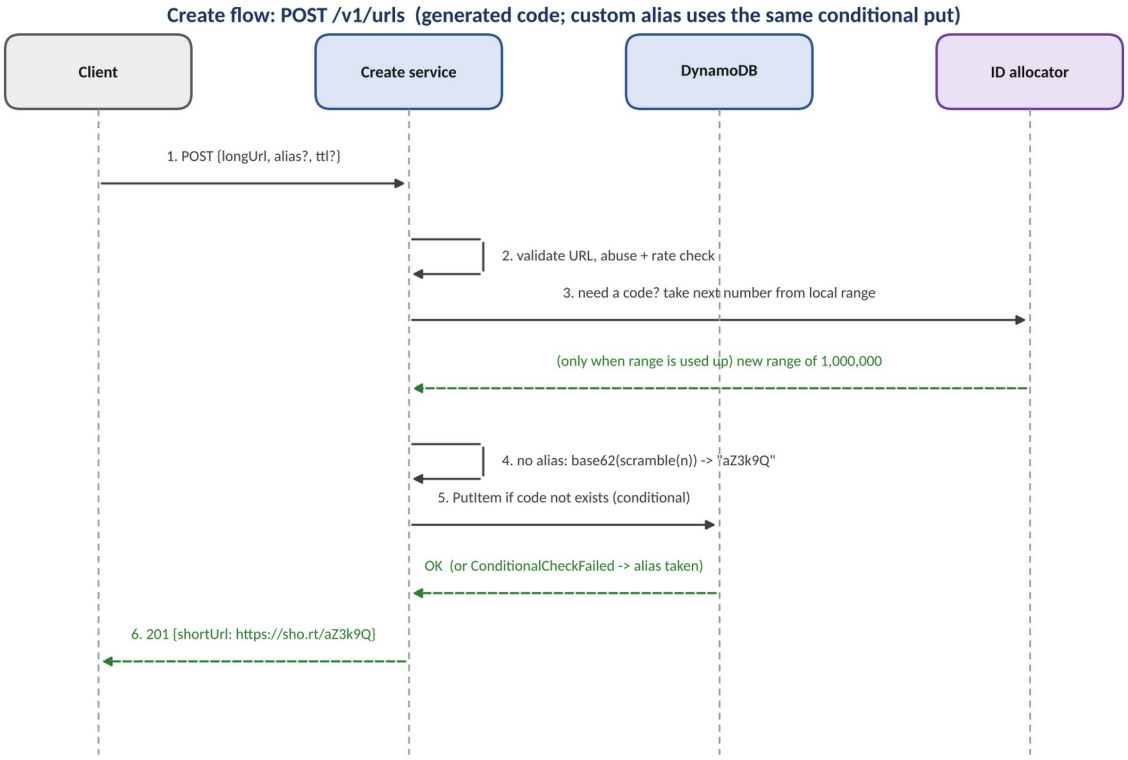


Figure 3. Create flow

1. The client sends `POST /v1/urls` with the long URL (and optionally an alias and expiry).
2. The Create service validates the URL (scheme, length, not on the blocklist) and applies a per-user rate limit.
3. If the user gave no alias, the service takes the next number from the range it already holds in memory. If its range is used up, it asks the allocator for a new block of 1,000,000 numbers (one DynamoDB call per million creates).
4. The number is scrambled and encoded in base62, producing for example `aZ3k9Q`. (Section D1 shows how.)
5. The service writes the mapping with a **conditional put** ("only if this code does not exist"). If it fails, the code is taken: for a generated code, pick the next number; for a custom alias, return 409.
6. The service returns 201 with the short URL. Nothing else is on the critical path.

C3. The read path in motion

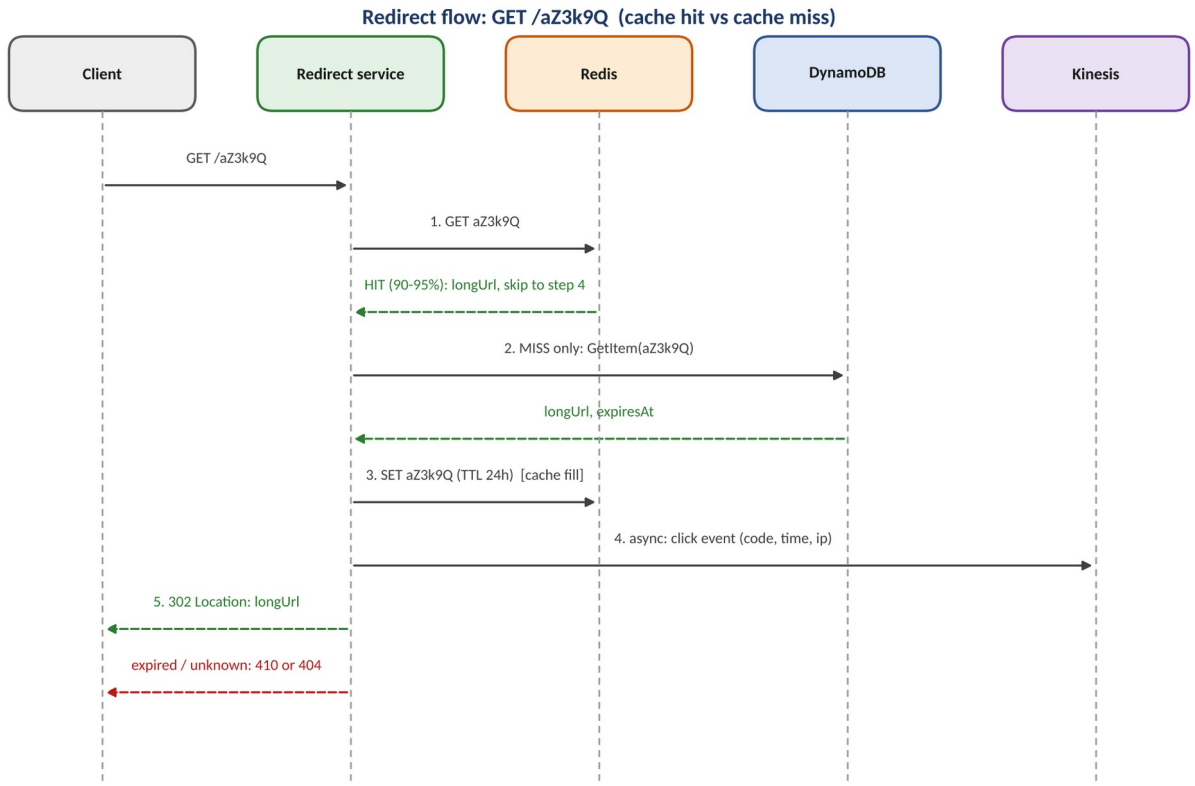


Figure 4. Redirect flow (cache hit and cache miss)

1. The browser requests `GET /aZ3k9Q`. The load balancer sends it to any Redirect instance.
2. The service looks the code up in Redis. On a **hit** (about 90 to 95 percent of requests) it has the long URL in under a millisecond.
3. On a **miss** it reads DynamoDB, checks `expiresAt`, writes the mapping into Redis with a TTL (cache-aside), and continues.
4. It publishes a click event (code, timestamp, country, user agent) to Kinesis asynchronously. If the stream is slow, the redirect is **not** delayed.
5. It returns `302 Found` with `Location: <long URL>`. If the code is unknown it returns 404; if expired, 410.

C4. The two-minute HLD talk track

Practice saying this out loud while drawing. It should take about two minutes and cover the diagram in the order of the data flow.

> **Say it like this:** "Clients reach us through a CDN and load balancer. I split the service into a Create service and a Redirect service because their traffic differs by about 100 to 1 and they have different failure impact: if creates are down, people cannot make links, but if redirects are down, every existing link on the internet is broken. Create gets unique codes from a range allocator, so there is no per-request coordination, then writes to DynamoDB with a conditional put so two requests can never claim the same code. Redirect checks Redis first; about 90 percent of requests end there. On a miss it reads DynamoDB and fills the cache. Click events go to a stream and are processed asynchronously for analytics, so counting clicks never slows a redirect. DynamoDB is replicated across availability zones, and I can add a second region later. The first things I would monitor are redirect latency, cache hit ratio and error rate."

> **Drawing order.** Client, load balancer, Create service, Redirect service, DynamoDB, then cache, then the allocator, then the stream. After each box say one sentence about why it exists. Never draw a box you cannot justify.

Part D. Deep dives with worked examples

D1. Generating the short code

This is the question the interviewer is most likely to push on. First learn the encoding, then compare the generation strategies.

Base62 encoding. Think of it as writing a number in base 62 instead of base 10. The alphabet is `0-9`, `a-z`, `A-Z`, so digit values 0 to 9 are the digits, 10 to 35 are lowercase letters and 36 to 61 are uppercase letters. Divide the number by 62 repeatedly and record the remainders; read them from last to first.

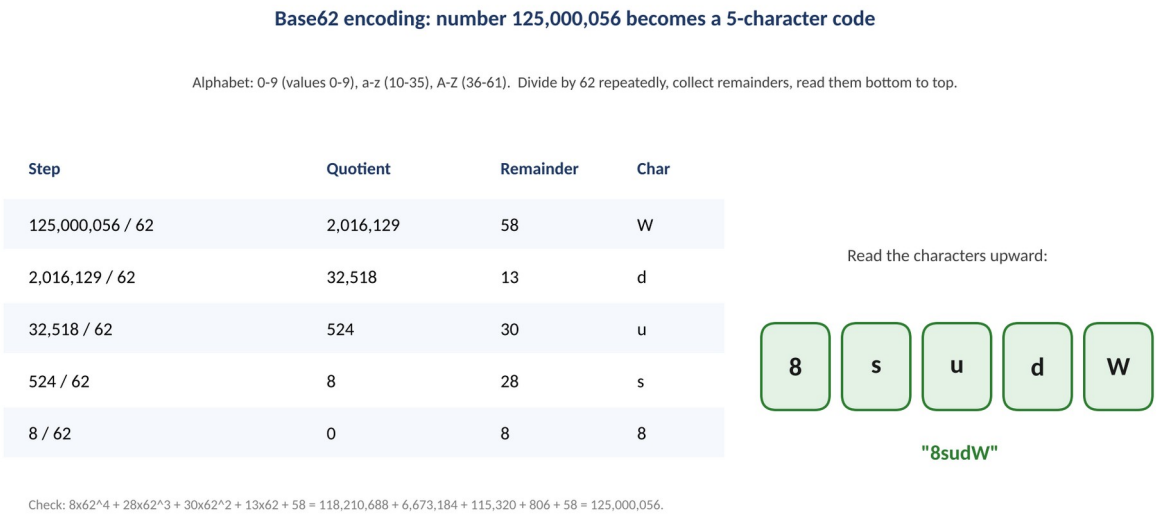


Figure 5. Encoding 125,000,056 in base62 gives "8sudW"

Decoding is the reverse: multiply each digit by its power of 62 and add. Both directions are O(length) and trivial to code. In Java:

```
private static final String ALPHABET =
        "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

static String encode(long n) {
    if (n == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (n > 0) {
        sb.append(ALPHABET.charAt((int) (n % 62)));
        n /= 62;
    }
    return sb.reverse().toString();
}
```

Now the strategies for choosing the number or string to encode.

Strategy	How it works	Problem	Verdict
Hash the long URL (for example MD5 or SHA-256), keep the first 7 characters	Same URL always gives the same code; no counter needed	Truncation causes collisions; two users shortening the same URL share a code (bad for per-user analytics or expiry); needs a retry loop with a salt	Acceptable fallback
Random 7 characters, retry on collision	Pick 7 random base62 symbols, attempt a conditional put, retry if it exists	About 1 in 590 inserts retries today and the rate grows as the table fills; one extra DB round trip on retry	Good and simple

Strategy	How it works	Problem	Verdict
Single counter plus base62	One database sequence gives 1, 2, 3...	Central bottleneck and single point of failure; sequential codes are guessable and enumerable	Do not use alone
Range allocation plus scramble plus base62	Each server leases a block of numbers; counts locally; scrambles; encodes	Gaps when a server dies (harmless); needs the scramble step to hide order	**Recommended**

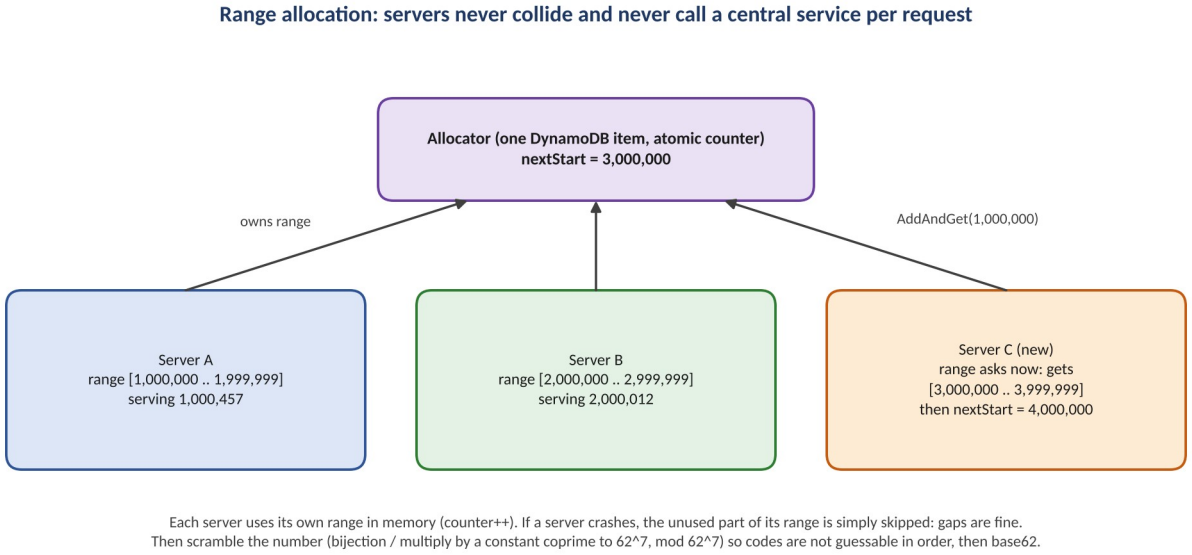


Figure 6. Servers lease number ranges, so creates need no per-request coordination

Worked example. Server A holds the range 1,000,000 to 1,999,999 and Server B holds 2,000,000 to 2,999,999. A user hits Server A: A takes 1,000,457 from memory, no network call. At the same instant a user hits Server B: B takes 2,000,012. They can never produce the same number because their ranges do not overlap. When Server C starts, it asks the allocator, which does one atomic increment of 1,000,000 on a single DynamoDB item and returns 3,000,000 to 3,999,999.

Why scramble? If you encode 1,000,457 directly you get codes that increase one by one, so a visitor can enumerate other people's links by incrementing the code. To hide the order, multiply the number by a large constant and take it modulo 62^7: `scrambled = (n x K) mod 62^7`. This is a bijection (every n gets a different result) as long as K shares no factor with 62^7, whose prime factors are 2 and 31. Any large K that is odd and not a multiple of 31, such as 2654435761, works. The result is below 62^7, so it fits in 7 characters (pad on the left with `0` to a fixed length) but looks random.

> **Say this if asked "what if two servers get the same number?"** They cannot: ranges are leased with an atomic increment, so each block is handed out exactly once. And as a second line of defense the database write is a conditional put, so even a bug in the allocator cannot overwrite an existing mapping.

D2. 301 or 302?

	301 Moved Permanently	302 Found (temporary)
Browser behavior	Caches the redirect; later clicks skip your servers	Asks your servers every time
Load on your system	Lower	Higher (every click arrives)
Click analytics	Lost for repeat visits from the same browser	Complete

	301 Moved Permanently	302 Found (temporary)
Expiry, deletion, destination changes	Cannot be enforced for browsers that cached it	Enforced immediately
Use when	Links are permanent and nobody needs click counts	You sell analytics or support expiry and delete (usually the case)

> **Say it like this:** "I would return 302. It costs more traffic, but we get complete analytics and we can honor expiry and deletion immediately. If cost became the concern, I would add short-lived edge caching in the CDN rather than switch to 301."

D3. Caching in detail

The cache is what lets a modest database serve 40,000 redirects per second. Use **cache-aside**: the service owns the logic. On a read: check the cache, on a miss read the database and fill the cache. Links are almost never updated, so there is little invalidation to worry about; on delete, remove the cache key.

Decision	Choice	Reason
Eviction	LRU (least recently used)	A link that was clicked recently is likely to be clicked again; matches the 80/20 pattern
Cache TTL	24 hours, with random jitter of plus or minus 10 percent	Bounds staleness after a delete; jitter prevents many keys expiring together (stampede)
Expired links in cache	Store `expiresAt` in the cached value and check it on every read	The cache must never keep serving a link that has expired
Negative caching	Cache "code does not exist" for about 60 seconds	Protects the database from bots probing random codes
Viral links	Let the CDN cache the 302 for 30 to 60 seconds for a very hot code	One link with 100,000 clicks per second would otherwise hammer a single cache shard

Hit-ratio example. At 40,000 reads per second with a 95 percent hit ratio, only 2,000 reads per second reach DynamoDB. At 80 percent it would be 8,000. This is why monitoring the hit ratio matters: a drop from 95 to 80 percent quadruples database load.

D4. Custom aliases and the race condition

Problem: two users submit the alias `promo` at the same moment. A naive "check if exists, then insert" lets both checks pass, and the second insert silently overwrites the first, sending the first user's visitors to the wrong site.

Time	User A	User B	Naive check-then-write	Conditional put
t0	checks "promo": not found	checks "promo": not found	both think it is free	(no separate check)
t1	writes promo -> site A	writes promo -> site B	B overwrites A. **Bug**	First write wins
t2				Second write fails with ConditionalCheckFailed, returns 409

The fix is to make the uniqueness check **part of the write**. In DynamoDB that is `PutItem` with the condition `attribute_not_exists(code)`. It is atomic on the server, so exactly one request wins. Also reserve certain words (`admin`, `login`, `api`) so users cannot claim paths you need.

D5. Expiry and deletion

- **Store `expiresAt`** with the mapping. DynamoDB TTL removes expired items in the background, but deletion can lag by up to roughly two days, so **always check `expiresAt` on read** and return 410 Gone when it has passed.
- **Reuse of codes:** do not recycle an expired code immediately; someone may still hold the old link. With a keyspace of 3.5 trillion there is no need to reuse at all.
- **Delete:** remove the DynamoDB item and the cache key. Because redirects are 302, browsers do not keep the old target.

D6. Analytics without slowing the redirect

Counting clicks directly in the database on every redirect would turn a read-only path into a write-heavy one and create hot partitions for popular links. Instead, the Redirect service publishes a small event to Kinesis or Kafka and returns immediately. Consumers aggregate the events (clicks per code per minute, by country and device) and store the totals in S3 or a columnar store for dashboards. If the stream is unavailable, drop or buffer the events: **losing a few click counts is acceptable, delaying a redirect is not**.

D7. Abuse and security

- **Rate limit creation** per user and per IP (this is Question 2 in action).
- **Check the destination** against a malware and phishing blocklist at creation time, and re-scan periodically, because a destination can turn malicious later.
- **Do not leak order:** the scramble step prevents enumeration of sequential codes.
- **Open redirect risk:** only redirect to links stored in your database; never to a URL taken from the request.

Part E. One complete example from start to finish

Putting every piece together with concrete values. Use a story like this to rehearse; it makes the explanation memorable.

Step	What happens	Components touched
1	Alice shortens `https://www.example.com/products/phones?id=88231`. No alias, no expiry.	Client, Load balancer
2	Create service (Server A, range 1,000,000 to 1,999,999) validates the URL and passes the rate limit. It takes n = 1,000,457.	Create service (memory only)
3	It computes (1,000,457 x 2654435761) mod 62^7, then encodes in base62 giving a 7-character code such as `aZ3k9Q1`.	Create service
4	Conditional put into DynamoDB succeeds. Alice receives `https://sho.rt/aZ3k9Q1`.	DynamoDB
5	Bob taps the link on his phone. Redirect service checks Redis: **miss** (first click).	Redirect service, Redis
6	It reads DynamoDB (about 5 ms), fills Redis with 24 h TTL, publishes a click event, and returns 302.	DynamoDB, Redis, Kinesis
7	Carol taps the same link a minute later: Redis **hit** (under 1 ms). 302 returned. Another click event is published.	Redis, Kinesis

Step	What happens	Components touched
8	Alice deletes the link. The service deletes the DynamoDB item and the Redis key.	DynamoDB, Redis
9	Dave taps it afterwards: Redis miss, DynamoDB miss, the service returns 410 Gone (or 404 if the item was already purged).	Redirect service

Part F. Reliability, scaling and trade-offs

F1. What happens when something fails

Failure	Effect	Mitigation
One Redirect or Create instance dies	Brief errors on in-flight requests	Stateless instances behind a load balancer with health checks; a dead Create instance only wastes the rest of its number range
A Redis node dies	Hit ratio drops; more load on DynamoDB	Replicas with automatic failover; DynamoDB is provisioned or on-demand to absorb the burst
Whole cache cluster down	Database takes full 40,000 reads per second at peak	On-demand capacity, plus the CDN edge cache; rate limit and circuit breaker to protect the database
DynamoDB slow in one availability zone	Higher latency	Multi-AZ by default; timeouts and retries with backoff; serve from cache while it lasts
Allocator unavailable	Cannot lease a new range	Each server holds a range with plenty left (1M numbers is about 7 hours at 40 creates per second per server) and prefetches the next range early
Kinesis unavailable	Click events lost or delayed	Local buffer, drop on overflow; redirects are never blocked
A region fails	Service unavailable in that region	DynamoDB global tables and a second region behind DNS failover; codes remain unique if each region leases ranges from a disjoint pool

F2. If traffic grows 10x

- **Redirect service:** add instances (stateless). It is the first thing to scale.
- **Cache:** add shards and replicas; use the CDN for very hot codes.
- **DynamoDB:** scales by partition; random codes distribute evenly, so there are no hot partitions except for a viral key, which the cache and CDN absorb.
- **Create path:** even at 4,000 creates per second the range allocator is called only about 4 times per second across the fleet.

F3. Trade-offs to state

Decision	Chosen	Alternative and its cost
Code generation	Range allocation plus scramble	Random plus retry is simpler but adds retries as the table fills; hashing couples code to URL
Redirect type	302	301 reduces traffic but loses analytics and control

Decision	Chosen	Alternative and its cost
Database	DynamoDB (key-value)	A relational database works at this scale with replicas, but needs more operational care at 40,000 reads per second
Analytics	Asynchronous via a stream	Synchronous counters are exact but slow and create hot rows
Consistency	Strong on create (conditional put), eventual on analytics	Strong everywhere adds cost with no user benefit

Part G. Follow-up questions with model answers

Interviewer asks	Strong answer
How do you generate unique codes across many servers?	Range allocation: each server leases a block of one million numbers with one atomic DynamoDB increment, counts locally, scrambles with a multiplier coprime to 62^7, and encodes base62. A conditional put is the safety net.
What if the same long URL is submitted twice?	By default create a new code each time, because owners, expiry and analytics are per link. If you want deduplication, keep an index of hash(longUrl + ownerId) to code and return the existing one.
How would you support 100 times more traffic?	Scale the stateless Redirect tier horizontally, add cache shards, push hot links to the CDN, and let DynamoDB on-demand absorb misses. Storage barely changes.
What if a single link goes viral?	It is one key, so it concentrates on one cache shard. Cache it at the CDN for 30 to 60 seconds and keep an in-process cache on each Redirect instance for the hottest codes.
How do you delete expired links?	DynamoDB TTL deletes in the background; reads check `expiresAt` anyway, so correctness never depends on cleanup timing.
How do you prevent abuse?	Rate limit creation, check destinations against blocklists, scramble codes to prevent enumeration, and offer a report-abuse flow that disables a code quickly.
SQL or NoSQL, and why?	Access is purely by key, there are no joins, volume is high and records are small, so a key-value store fits. A relational database would also work for the first stage.
How would you test the system?	Unit-test encode and decode with round trips; load-test redirects with a realistic hot-key distribution; chaos-test cache and allocator outages; verify no duplicate codes under concurrent creates.

Part H. Common mistakes and a one-page cheat sheet

- **Jumping straight to the database schema** without clarifying scale or requirements.
- **Using one auto-increment counter** and never explaining the bottleneck or guessable codes.
- **Saying "hash the URL"** without handling collisions or the same URL submitted by two users.
- **Forgetting 301 vs 302** and what it does to analytics and expiry.
- **Ignoring the read/write ratio:** designing both paths identically instead of optimizing the read path.
- **No failure story:** not saying what happens when the cache or the allocator is down.

Cheat sheet	Answer
Core insight	Read-heavy (100:1), small records, key-value access
Numbers	40 writes/s (400 peak), 4,000 reads/s (40,000 peak), 6 B records, about 3 TB, about 35 GB cache
Code	7 characters of base62 (3.5 trillion codes); range allocation, scramble, base62
Storage	DynamoDB, partition key = code, conditional put, TTL for expiry
Read path	CDN, load balancer, Redirect service, Redis (90 to 95 percent hits), DynamoDB on miss, 302
Async	Kinesis for clicks; never block a redirect
Risks	Hot link, cache outage, alias race, abuse

Question 2: Design a Rate Limiter

> **How to use this chapter.** Part A explains the idea in plain language. Part B covers requirements and numbers. Part C teaches each algorithm with a worked example. Part D is the single-node code. Part E is the distributed High-Level Design (HLD) you will draw and explain. Part F covers the deep dives interviewers push on, and Part G is rehearsal material.

Part A. Understand the problem first

A rate limiter decides, for every incoming request, whether to let it through or reject it because the caller has sent too many requests in a given period. Real examples: an API that allows 100 requests per minute per API key; a login endpoint that allows 5 attempts per minute per IP address; a payment API that allows 10 requests per second per merchant.

Why companies build one

- **Protect the backend** from overload, whether from a bug (a client stuck in a retry loop) or an attack.
- **Fairness:** one noisy customer must not slow down everybody else.
- **Cost control:** every request costs compute, database reads and sometimes money paid to a third party.
- **Abuse prevention:** slow down password guessing, scraping and spam.
- **Monetization:** free tier gets 100 requests per minute, paid tier gets 10,000.

Where it can live

Placement	Pros	Cons
Client side	No server cost	Cannot be trusted; clients can ignore it
API gateway or sidecar (recommended)	One place to enforce, protects all services, rejects early before expensive work	Needs shared state when there are many gateways
Inside each service	Can use business context (for example per-tenant plans)	Duplicated logic; the request has already used resources

> **Key idea.** A rate limiter is a tiny decision made billions of times: "has this client used up its allowance?" The design challenge is making that decision **fast** (a millisecond or less), **correct under concurrency** (two servers deciding at once), and **safe** (it must never become the reason the API goes down).

Part B. Requirements and numbers

B1. Clarifying questions

Question to ask	Why it matters	Assumption to state
Where is the limiter placed?	Decides the architecture	At the API gateway protecting a public API
What identifies a client?	Decides the key	API key for authenticated calls; IP address for anonymous ones
Hard limit or soft throttling?	Reject versus slow down	Hard limit: reject with HTTP 429
Are small bursts allowed?	Chooses the algorithm	Yes: allow short bursts above the steady rate

Question to ask	Why it matters	Assumption to state
Different limits for different endpoints or tiers?	Needs a rules system	Yes, rules are configuration, not code
What if the limiter itself fails?	Availability versus protection	Fail open for most endpoints, fail closed for a few sensitive ones
Single region or global limit?	Cross-region coordination is costly	Single region first, multi-region as an extension

> **Say it like this:** "I will build a distributed rate limiter at the API gateway. Clients are identified by API key or IP, limits are per endpoint and per plan tier, bursts are allowed, rejected calls get HTTP 429 with a Retry-After header, and if the limiter store is unavailable we default to allowing traffic so the limiter never takes the API down."

B2. Back-of-the-envelope estimates

Quantity	Working	Result
Peak traffic	Assumption given or chosen: 1 million API requests per second across all gateways	1 million limiter checks per second
Active clients	Assumption: 50 million distinct keys active in the window	50 million buckets
State per client	key (about 40 B) + tokens (8 B) + timestamp (8 B) + overhead, about 100 B	100 bytes
Total state	50 million x 100 bytes	about 5 GB (fits in memory)
Redis capacity	One node handles roughly 100,000 simple operations per second	1M / 100k = 10 shards minimum; plan 20 for headroom and failover
Added latency	One in-datacenter round trip	about 0.5 to 1 ms per request

> **What the numbers tell you.** The state is small (5 GB) but the **operation rate** is large (1 million per second), so the store must be an in-memory system and it must be sharded for throughput, not for size. Say this.

B3. Rules and the response a client sees

```
// Rule (stored in a config service, cached by gateways)
{ "ruleId": "search-api", "limit": 100, "windowSeconds": 60, "burst": 20 }

// When allowed: normal response plus informational headers
HTTP 200 OK
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 37

// When blocked
HTTP 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
```

The `Retry-After` header tells well-behaved clients when to try again, which prevents them from hammering the API in a retry loop. Good clients combine it with exponential backoff and jitter.

Part C. The algorithms, each with a worked example

Interviewers expect you to know at least four algorithms, to state the weakness of each, and to choose one with a reason. Learn them in this order, because each one fixes the flaw of the one before.

C1. Fixed window counter

Divide time into fixed windows (for example each calendar minute). Keep one counter per client per window. Each request increments the counter; if it exceeds the limit, reject. At the start of the next window the counter resets.

Example. Limit 100 per minute. A client sends 100 requests between 12:00:00 and 12:00:59: all allowed, the 101st is rejected. At 12:01:00 the counter resets to zero.

Weakness: boundary bursts. A client can send 100 requests in the last seconds of one window and 100 more in the first seconds of the next, so 200 requests pass in a few seconds, double the intended rate.

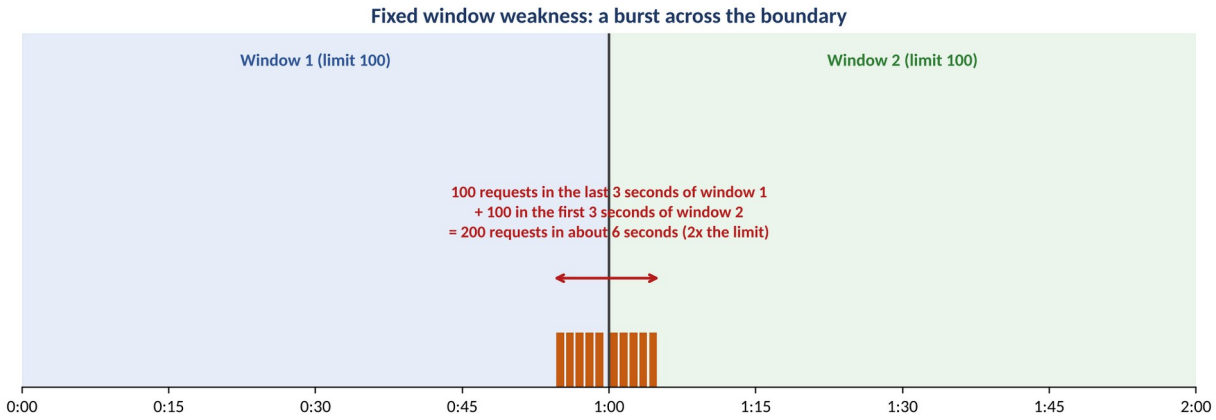


Figure 1. Fixed window allows 2x the limit across a boundary

Strength: extremely simple, O(1) memory (one counter), one atomic `INCR` plus `EXPIRE` in Redis. Use it when approximate is fine.

C2. Sliding window log

Store the timestamp of every accepted request. On a new request, discard timestamps older than the window, count the rest, and accept only if the count is below the limit.

Example. Limit 3 per 10 seconds.

Time (s)	Log before (after dropping entries older than now minus 10)	Count	Decision	Log after
1	[]	0	Allow	[1]
3	[1]	1	Allow	[1, 3]
6	[1, 3]	2	Allow	[1, 3, 6]
9	[1, 3, 6]	3	**Reject** (limit 3)	[1, 3, 6]
11.5	drop entries before 1.5: [3, 6]	2	Allow	[3, 6, 11.5]

Strength: exact, no boundary problem. Weakness: memory is proportional to the limit per client (a limit of 10,000 per hour means up to 10,000 timestamps per client), and counting is more work. With 50 million clients that becomes impractical.

C3. Sliding window counter (approximation)

Keep only two counters per client: the previous window and the current window. Estimate the number of requests in the sliding window by weighting the previous window by how much of it still overlaps.

```
estimate = previousCount x overlapFraction + currentCount
```

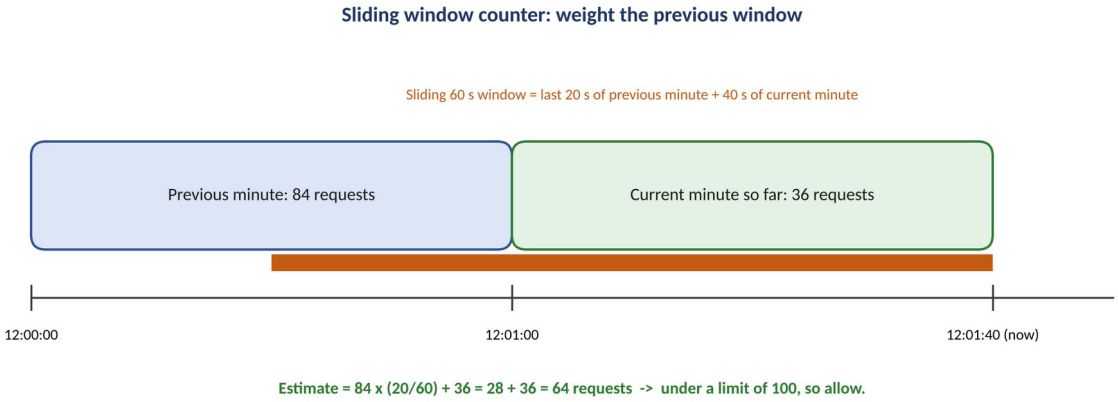


Figure 2. Sliding window counter: 84 x (20/60) + 36 = 64 requests

Worked example. It is 12:01:40, so we are 40 seconds into the current minute and the sliding window still covers the last 20 seconds of the previous minute (overlap 20/60 = one third). The previous minute had 84 requests, the current minute so far has 36. Estimate = 84 x 1/3 + 36 = 28 + 36 = 64. Under a limit of 100, allow. The result assumes the previous window's requests were spread evenly; that assumption is the source of a small error, but in practice the approximation is very close and memory is tiny.

C4. Token bucket (the one to recommend)

Picture a bucket that holds up to **capacity** tokens. Tokens are added at a steady **refill rate** (for example one per second), but the bucket never holds more than its capacity. Each request must take one token. If a token is available the request passes; if the bucket is empty the request is rejected.

- **Capacity** controls the maximum burst (how many requests can arrive at once).
- **Refill rate** controls the sustained average rate.
- **State per client** is just two numbers: the token count and the time of the last refill.
- **Lazy refill:** there is no timer. When a request arrives, compute how many tokens should have been added since the last request: `tokens = min(capacity, tokens + elapsedSeconds x refillRate)`. This is why it scales to millions of clients.

Worked example. Capacity 5, refill 1 token per second, bucket starts full (5.0). Follow the numbers:

Time (s)	Tokens before refill	Refill since last	Tokens after refill	Decision	Tokens after request
0.0	5.0 (full)	0	5.0	Allow	4.0
0.5	4.0	+0.5	4.5	Allow	3.5
1.0	3.5	+0.5	4.0	Allow	3.0
1.2	3.0	+0.2	3.2	Allow	2.2
1.3	2.2	+0.1	2.3	Allow	1.3
1.4	1.3	+0.1	1.4	Allow	0.4

Time (s)	Tokens before refill	Refill since last	Tokens after refill	Decision	Tokens after request
1.5	0.4	+0.1	0.5	**Reject** (needs 1.0)	0.5
2.0	0.5	+0.5	1.0	Allow	0.0
2.1	0.0	+0.1	0.1	**Reject**	0.1

Notice the behavior: six requests were accepted in the first 1.4 seconds (a burst, spending the saved-up tokens), then the limiter throttled to roughly one request per second. At t = 1.5 the client has 0.5 tokens and needs 1.0, so it would be told `Retry-After: 1` (the number of seconds until one full token exists, rounded up: (1 - 0.5) / 1 = 0.5, rounded to 1).

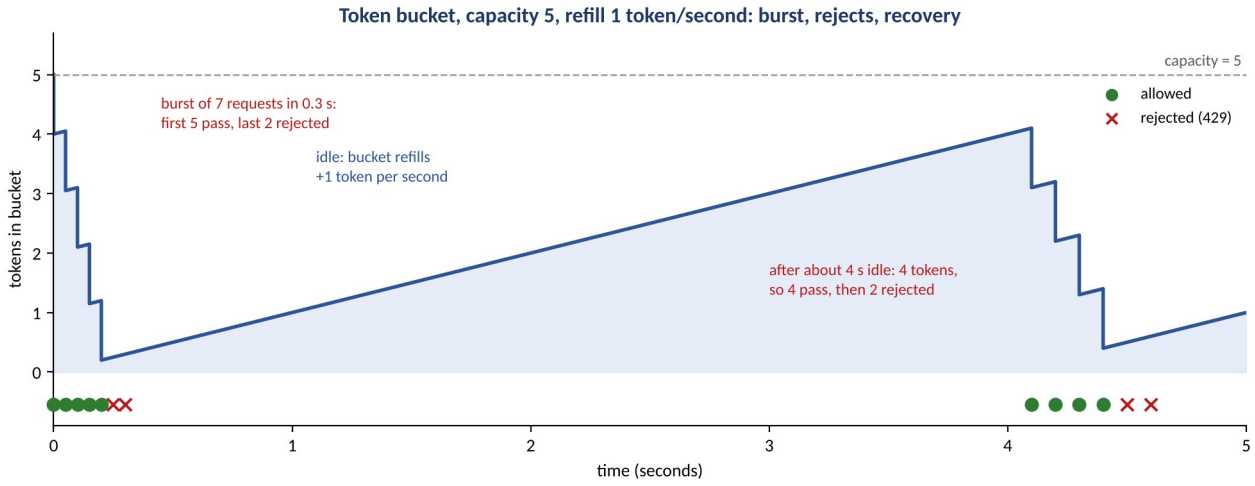


Figure 3. Token bucket: a burst is allowed, then rejected, then the bucket recovers while idle

This chart shows capacity 5 and refill 1 per second: a burst of 7 requests in 0.3 seconds is accepted for the first 5 and rejected for the last 2; after about 4 idle seconds the bucket holds about 4 tokens, so the next 4 pass and the rest are rejected.

C5. Leaky bucket (know it, usually do not choose it)

Requests enter a queue and are processed (leak out) at a fixed rate. If the queue is full, new requests are dropped. It produces a perfectly smooth output rate, which suits protecting a downstream system that needs a constant flow, but it **delays** requests instead of rejecting them quickly, and a burst fills the queue with old requests. For an API gateway, token bucket is usually the better fit.

C6. Comparison and the recommendation

Algorithm	Memory per client	Accuracy	Allows bursts?	Main weakness
Fixed window counter	1 counter	Poor at boundaries (up to 2x)	Accidentally	Boundary burst
Sliding window log	N timestamps	Exact	No	Memory and cost
Sliding window counter	2 counters	Very close	Limited	Assumes even spread
Token bucket	2 numbers	Exact for the model	**Yes, controlled**	Two parameters to tune
Leaky bucket	Queue	Exact output rate	Smooths bursts	Adds delay

> **Say it like this:** "I would use a token bucket. Real clients send short bursts, and a token bucket lets me allow a controlled burst while enforcing the long-run average. It needs only two numbers per client, refills lazily so there are no timers, and it can be updated atomically in one Redis call."

Part D. Single-node implementation (the low-level design half)

If the interviewer asks for classes and code first, this is the answer. It is Java 8 compatible and has been compiled and run: with capacity 5 and refill 10 per second, a burst of 5 requests is all accepted and the sixth is rejected, and after 0.5 seconds about 5 tokens are available again.

```
class TokenBucket {
    private final long capacity;
    private final double refillPerNano;   // tokens added per nanosecond
    private double tokens;
    private long lastRefillNanos;

    TokenBucket(long capacity, double refillPerSecond) {
        this.capacity = capacity;
        this.refillPerNano = refillPerSecond / 1_000_000_000.0;
        this.tokens = capacity;
        this.lastRefillNanos = System.nanoTime();
    }

    synchronized boolean tryConsume() {
        long now = System.nanoTime();
        tokens = Math.min(capacity, tokens + (now - lastRefillNanos) * refillPerNano);
        lastRefillNanos = now;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }
}

class RateLimiter {
    private final Map<String, TokenBucket> buckets = new ConcurrentHashMap<>();
    private final long capacity;
    private final double refillPerSecond;

    RateLimiter(long capacity, double refillPerSecond) {
        this.capacity = capacity; this.refillPerSecond = refillPerSecond;
    }
    boolean allowRequest(String clientId) {
        return buckets.computeIfAbsent(clientId,
                k -> new TokenBucket(capacity, refillPerSecond)).tryConsume();
    }
}
```

Line-by-line reasoning to say aloud

- **`refillPerNano` and `System.nanoTime()`:** a monotonic clock, unaffected by someone changing the system time. Use it for measuring elapsed time, never `currentTimeMillis`.
- **Lazy refill in `tryConsume`:** the first line computes how many tokens accrued since the last call and caps the result at capacity. No background thread is needed per client.
- **`synchronized` on the bucket:** the read-refill-decrement sequence must be atomic or two threads could both spend the same token. The lock is per bucket, so different clients never block each other.
- **`ConcurrentHashMap.computeIfAbsent`:** creates a bucket exactly once per client even when two threads arrive simultaneously.

- **Memory:** idle buckets must be evicted (a scheduled cleanup or an LRU map), otherwise a million one-time visitors leave a million buckets in memory forever.
- **Testability:** to unit-test refill math, inject a clock (a `LongSupplier` for time) instead of calling `System.nanoTime()` directly, so a test can advance time without sleeping.

> **Why single-node code is not enough.** This works inside one process. With 20 gateway servers behind a load balancer, each server has its own map, so a client with a limit of 100 can send 100 to each server: 2,000 requests, not 100. The rest of this chapter is about fixing that.

Part E. High-Level Design (HLD) for the distributed limiter

E1. Why local counters fail

Suppose the limit is 100 per minute and there are 4 gateways. There are two naive options, and both break:

Naive option	What goes wrong	Example
Each gateway enforces the full limit locally	The real limit becomes limit x number of gateways	Client sends 100 requests to each of the 4 gateways: 400 pass
Each gateway enforces limit / number of gateways (25 each)	Load balancing is uneven, so legitimate clients are blocked early	A client whose requests all land on gateway 1 is blocked at 25 even though it used only 25 percent of its allowance
Sticky routing by client	Hot clients overload one gateway; failover loses state	One big customer pins one gateway at 100 percent CPU

The standard answer is **shared state in a fast central store**, with all gateways reading and updating the same bucket for a client.

E2. The architecture to draw

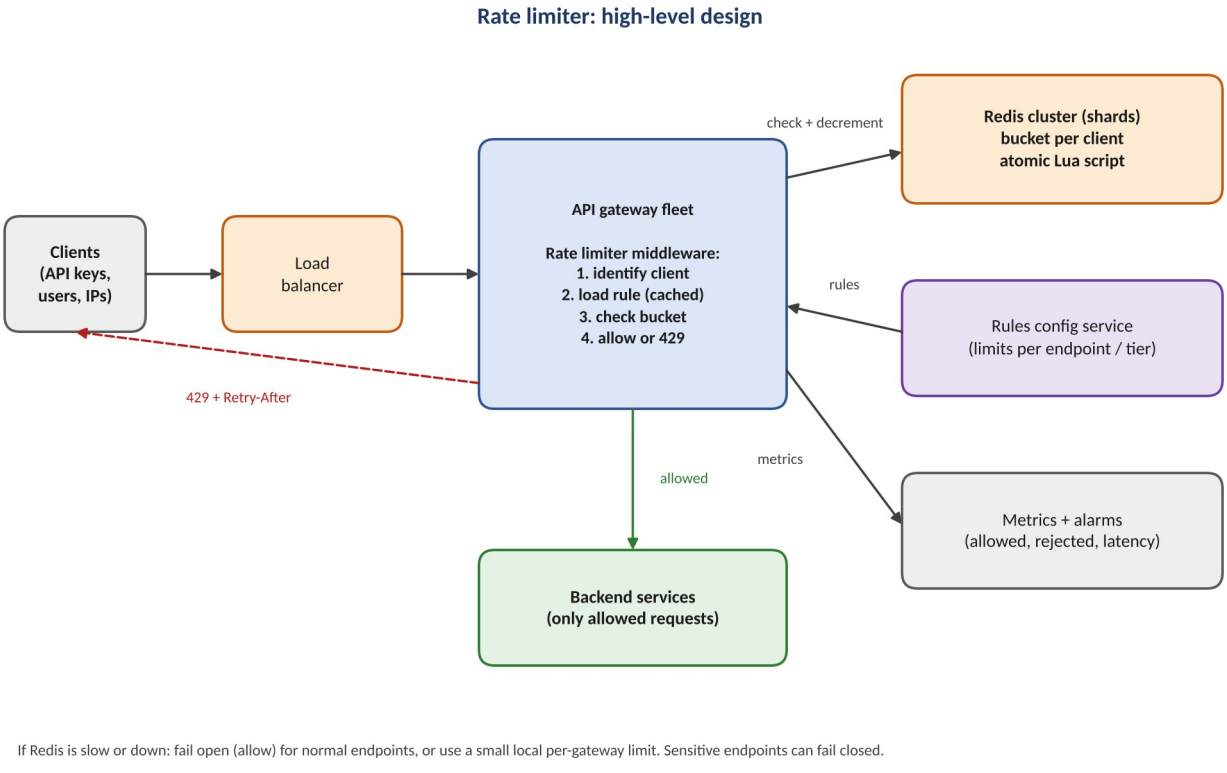


Figure 4. Rate limiter HLD

Component	Responsibility	Technology example	Why
Load balancer	Spread traffic across gateways	ALB / NLB	Standard front door
API gateway with limiter middleware	Identify the client, load the rule, call the store, allow or return 429	Gateway fleet (Envoy, Kong, custom service)	Enforcing at the edge rejects bad traffic before it costs backend resources
Shared counter store	Holds the bucket (tokens and timestamp) per client	Redis cluster (ElastiCache)	In-memory, sub-millisecond, atomic scripts
Rules configuration service	Stores limits per endpoint and tier; gateways cache it	DynamoDB or a config store with push or polling	Change limits without redeploying
Metrics and alarms	Allowed versus rejected per rule, limiter latency, store health	CloudWatch, Prometheus	You must see who is throttled and whether the limiter is healthy

E3. One request, step by step

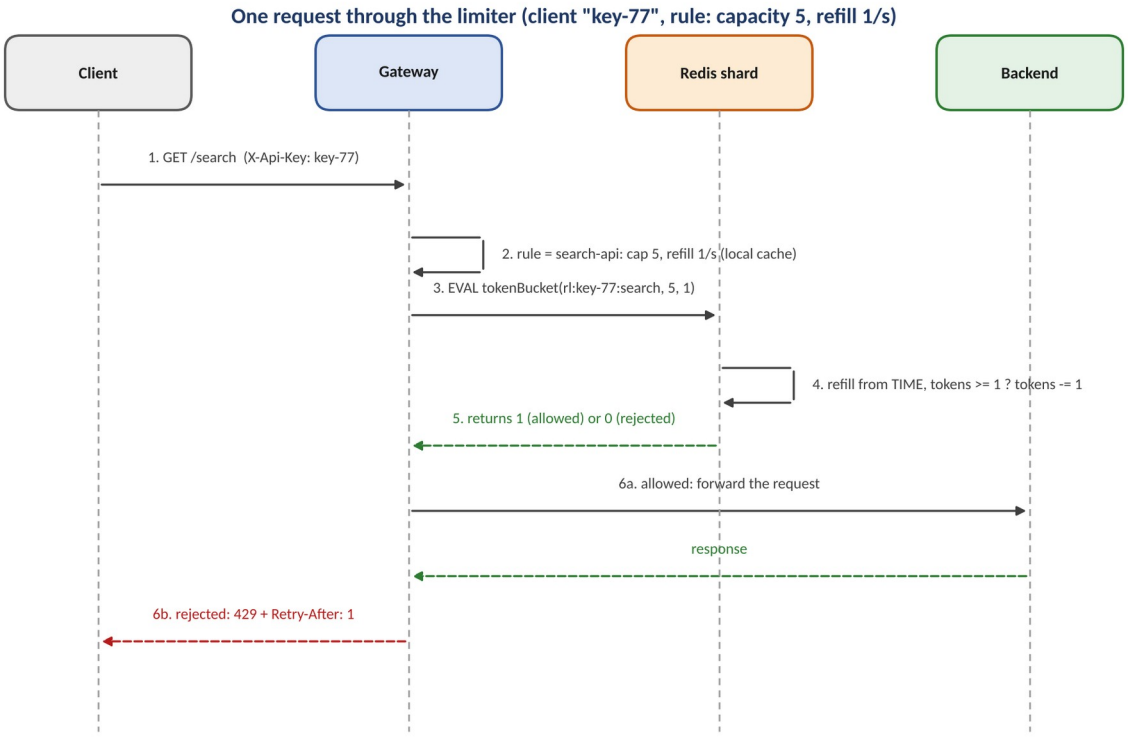


Figure 5. One request through the limiter

1. The request arrives at a gateway with `X-Api-Key: key-77`. The gateway extracts the client identity.
2. It looks up the rule for the endpoint (here: capacity 5, refill 1 per second). Rules are cached in memory and refreshed every few seconds.
3. It sends one atomic command to Redis for the key `rl:key-77:search` (the Lua script in the next section).
4. Redis refills the bucket from its own clock, tries to take one token, writes the new state and returns 1 (allowed) or 0 (rejected).
5. Allowed: the gateway forwards the request to the backend. Rejected: the gateway immediately returns 429 with `Retry-After` and never touches the backend.

6. The decision is counted in metrics, tagged with the rule and client tier.

E4. The race condition and why the Lua script solves it

Without atomicity, two gateways can both read the same state. Example with one token left:

Step	Gateway A	Gateway B	Redis tokens
1	reads tokens = 1		1
2		reads tokens = 1	1
3	tokens >= 1, so allow, writes 0		0
4		tokens >= 1 (stale read), so allow, writes 0	0
Result	Allowed	**Allowed (wrong)**	Two requests used one token

Redis runs a Lua script as one uninterruptible unit, so no other command interleaves between the read and the write. The check-and-decrement becomes one atomic operation:

```
-- KEYS[1] = bucket key; ARGV[1] = capacity; ARGV[2] = refill tokens per second
local t = redis.call('TIME')
local now = tonumber(t[1]) + tonumber(t[2]) / 1000000   -- Redis clock avoids skew
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local capacity = tonumber(ARGV[1])
local rate = tonumber(ARGV[2])
local tokens = tonumber(data[1]) or capacity
local ts = tonumber(data[2]) or now
tokens = math.min(capacity, tokens + (now - ts) * rate)
local allowed = 0
if tokens >= 1 then tokens = tokens - 1; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', KEYS[1], math.ceil(capacity / rate) * 2)  -- idle buckets vanish
return allowed
```

Walking through it. A brand-new client has no hash, so `tokens` defaults to the capacity (5) and `ts` to now. The script computes `tokens = min(5, 5 + 0 x 1) = 5`, takes one (4), saves it and returns 1. The next request 0.5 seconds later reads tokens 4 and ts, computes `min(5, 4 + 0.5 x 1) = 4.5`, takes one (3.5) and returns 1. The `EXPIRE` sets the key to vanish after twice the time needed to refill a full bucket, so a client that goes quiet costs no memory: a fresh bucket would be full anyway, so deleting an idle one loses nothing. Note: calling `TIME` inside a script that then writes requires Redis 5 or later (earlier versions disallow writes after a non-deterministic command unless script effects replication is enabled).

E5. Sharding the store

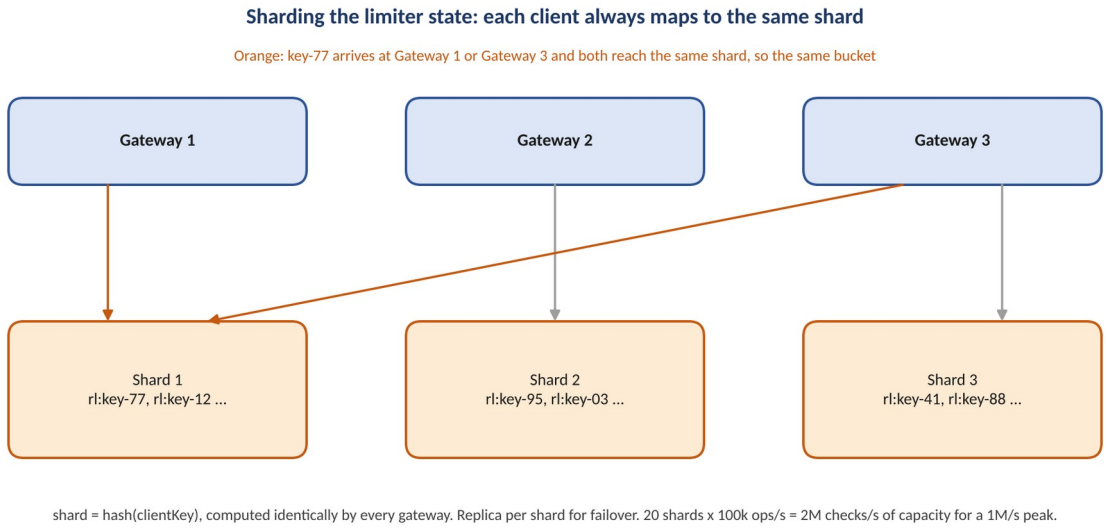


Figure 6. Every gateway maps a client key to the same Redis shard

All the state for one client lives on one shard, chosen by hashing the key (consistent hashing, or Redis Cluster hash slots). Every gateway computes the same hash, so they all reach the same bucket. A scripted operation touches one key, so it never needs a cross-shard transaction. Capacity example: 20 shards x 100,000 operations per second = 2 million checks per second, comfortably above the 1 million per second peak. Each shard has a replica; if a primary fails, the replica is promoted and buckets may briefly show slightly stale counts, which is acceptable for rate limiting.

E6. The two-minute HLD talk track

> **Say it like this:** "I put the limiter in the API gateway so rejected requests never consume backend resources. For each request the gateway identifies the client by API key, loads that endpoint's rule from a locally cached configuration, and makes one call to a Redis cluster that holds a token bucket per client. The call is a Lua script, so the read, refill, decrement and write are atomic, and it uses Redis's clock to avoid gateway clock skew. If a token is available the request is forwarded; otherwise the gateway returns 429 with Retry-After. Buckets are sharded by client key so each client has exactly one home shard, and each shard has a replica. If Redis is slow or down, I fail open for normal endpoints so the limiter cannot cause an outage, with a conservative local limit as a fallback, and fail closed for a few sensitive endpoints like login. I would monitor allowed versus rejected rate per rule, limiter latency and Redis health."

> **Drawing order.** Client, load balancer, gateway (write the four middleware steps inside it), Redis cluster, rules service, metrics. Then draw the 429 arrow back to the client. Finish by saying what happens if Redis fails.

Part F. Deep dives

F1. Reducing Redis traffic with local token batches

One Redis call per request is simple but costs 1 million calls per second and about a millisecond of latency. An optimization: each gateway asks Redis for a **batch of tokens** (for example 10) for a busy client, then serves the next 10 requests from memory with no network call.

	Per-request Redis call	Batch of 10 tokens
Redis operations at 1M requests per second	1,000,000 per second	about 100,000 per second
Added latency	0.5 to 1 ms on every request	near zero for 9 of every 10 requests

	Per-request Redis call	Batch of 10 tokens
Accuracy	Exact	Can over-admit by up to batch size x number of gateways, and tokens sit unused in one gateway while others starve
Good for	Strict limits (payments, login)	High-volume, tolerant limits (general API traffic)

The accuracy cost is easy to bound: with 20 gateways and a batch of 10, a client can briefly gain at most 200 extra tokens. Choose the batch size as a small fraction of the client's limit.

F2. When the limiter store fails: fail open or fail closed?

Choice	Behavior	Use for	Risk
Fail open	Allow all requests while the store is unreachable	General API traffic	No protection during the outage; backend may be overloaded
Fail closed	Reject all requests	Login, password reset, payment (abuse-sensitive endpoints)	The limiter outage becomes an API outage
Degraded mode (best)	Each gateway applies a conservative local in-memory limit, for example the global limit divided by the number of gateways times a safety factor	Most endpoints	Less accurate, but bounded protection and no outage

Also set a very short timeout on the Redis call (for example 5 to 10 ms) and use a circuit breaker, so a slow Redis does not add its delay to every request.

F3. Clock skew

Refill depends on elapsed time. If each gateway used its own clock and the clocks differ by 2 seconds, gateways would disagree about how many tokens have accrued, and a client could benefit from whichever gateway is "ahead". Reading the time inside the Redis script uses one clock for the whole shard and removes the problem.

F4. Multi-region

- **Strict global limit:** every check crosses regions (50 to 150 ms), which is unacceptable for the hot path.
- **Usual compromise:** split the global limit across regions (for example 60 percent / 40 percent based on traffic) and rebalance periodically. A client that spreads traffic evenly gets the full limit; a client that hammers one region is limited by that region's share.
- **Alternative:** asynchronously gossip usage between regions and accept brief over-admission.

F5. Rules, tiers and identification

- **Rules are data:** map plan tier and endpoint to (capacity, refill rate). Gateways cache rules and refresh them every few seconds; a new rule applies without a deploy.
- **Multiple limits at once:** for example per key (1,000 per minute) and per endpoint (50 per second for `/search`) and per IP; check all and reject if any fails. Store each under its own Redis key.
- **Client identity:** API key or user ID when authenticated; IP address for anonymous traffic. Behind a proxy or load balancer, read the real IP from `X-Forwarded-For` only from trusted proxies; otherwise attackers spoof the header to evade limits. Beware that many users can share one IP (offices, mobile carriers), so IP limits should be generous.

F6. What the client should do

A good API contract includes guidance for callers: respect `Retry-After`, use exponential backoff with jitter on retries, and avoid synchronized retries. Without jitter, 1,000 clients that were all rejected at the same moment all retry at the same moment and recreate the spike.

Part G. Trade-offs, follow-ups and cheat sheet

G1. Trade-offs to state

Decision	Chosen	Alternative and its cost
Algorithm	Token bucket	Sliding window log is exact but memory-heavy; fixed window is cheap but allows 2x bursts
State location	Central Redis with Lua script	Local-only counters are fastest but let a client exceed its limit by the number of gateways
Accuracy vs latency	Per-request check by default; batches for high-volume tolerant limits	Batches reduce load and latency but can over-admit slightly
Failure mode	Fail open or degraded local limit	Fail closed protects the backend but turns a limiter outage into an API outage
Placement	API gateway	Per-service limiting has business context but wastes resources on requests already accepted

G2. Follow-up questions with model answers

Interviewer asks	Strong answer
How do you rate limit across many servers?	Shared state in Redis, sharded by client key, updated with an atomic Lua script so two gateways cannot both spend the last token.
What if Redis goes down?	Fail open by default with a short timeout and circuit breaker; apply a conservative local limit per gateway as a fallback; fail closed only for abuse-sensitive endpoints; alarm immediately.
How do you handle bursts?	Token bucket: capacity sets the burst size and refill rate sets the sustained rate. Both are configurable per rule.
How do you limit by IP behind a proxy?	Trust X-Forwarded-For only from known proxy addresses and take the right-most untrusted entry; otherwise it can be spoofed.
How would you support different tiers?	Rules are configuration keyed by tier and endpoint; the gateway resolves the client to a tier and fetches the matching rule.
How do you keep Redis from becoming a bottleneck?	Shard by client key (20 shards for 2M ops per second), add replicas, and use local token batches for high-volume tolerant limits.
How would you test it?	Unit-test refill math with an injectable clock, run a concurrency test that fires many threads at one bucket and checks the allowed count never exceeds the limit, load-test the Redis tier, and test failure modes (store down, slow store).
How do you know it works in production?	Dashboards for allowed vs rejected by rule, p99 limiter latency, Redis CPU and memory, and alarms on rejection spikes and limiter errors.

G3. Common mistakes

- **Only describing the algorithm** and never addressing the distributed case. Most of the interview is the distributed part.
- **Read then write without atomicity.** Always name the race and the atomic fix.
- **Using gateway clocks** for refill instead of one authoritative clock.
- **No failure policy:** saying "Redis stores counters" without saying what happens when it is unavailable.
- **Forgetting memory cleanup:** idle buckets must expire (TTL in Redis, eviction in the single-node code).

G4. One-page cheat sheet

Cheat sheet	Answer
Core insight	Tiny decision, huge volume: must be fast, atomic and safe
Numbers	1M checks per second, 50M clients, about 5 GB state, 20 Redis shards, about 1 ms added latency
Algorithm	Token bucket: capacity = burst, refill rate = sustained rate, lazy refill, two numbers per client
Single node	Map of client to bucket, synchronized refill and consume, evict idle buckets
Distributed	Gateway middleware, Redis cluster with Lua, shard by key, rules in a config service; extras: local token batches, multi-region split limits, Redis clock
Response	429 with Retry-After and X-RateLimit headers
Failure	Fail open or local fallback; fail closed for sensitive endpoints; short timeout and circuit breaker

Next Steps

How to practice these two chapters

1. Read the chapter once for understanding. Do not memorize numbers; understand why each number exists.
2. Close the document and redraw the HLD from memory. Check it against the figure.
3. Say the two-minute HLD talk track aloud, timing yourself.
4. Pick three deep dives (for Q1: code generation, 302 vs 301, alias race; for Q2: algorithm choice, atomic Lua, fail-open) and explain each in two minutes.
5. Ask a friend to play interviewer using the follow-up tables, or answer them out loud and compare.

Questions 3 to 5 (e-commerce order and inventory, news feed, distributed cache) remain in the Top 5 guide and can be expanded in the same format next.